

1. Regresja/klasyfikacja na tekście, funkcje kosztu

- **Regresja na tekście:** Polega na przewidywaniu ciągłej wartości liczbowej (np. ocena sentymentu od 1 do 5, wycena nieruchomości na podstawie opisu).
- **Klasyfikacja na tekście:** Polega na przypisywaniu tekstom etykiet dyskretnych (np. spam / nie-spam, kategoria tematyczna artykułu).
- **Funkcje kosztu (Loss functions):** Służą do mierzenia błędu modelu podczas uczenia:
 - **Log loss (Mean Squared Error lub inne w zależności od zadania):** W modelach językowych powszechnie stosuje się funkcję straty opartą na negatywnym logarytmie prawdopodobieństwa (Log loss): $-\log(p_j)$, gdzie p_j to przypisane przez model prawdopodobieństwo dla poprawnego słowa w_j , które rzeczywiście wystąpiło w tekście. Model jest silnie karany, jeśli rzeczywistej klasie przypisze niskie prawdopodobieństwo.

2. Regresja liniowa na tekście

- Model regresji liniowej zakłada liniową zależność między cechami reprezentującymi tekst (np. częstościami słów) a ciągłą wartością wyjściową y .
- Równanie przyjmuje postać: $\hat{y} = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$, gdzie x_i to cechy tekstu (np. wartości TF-IDF słów), a w_i to wagi uczone przez model.

3. Regresja logistyczna na tekście

- Używana do zadań **klasyfikacji** (mimo słowa "regresja" w nazwie). Przekształca surowy wynik liniowy (tzw. logit) za pomocą funkcji sigmoidalnej $\sigma(z) = \frac{1}{1 + e^{-z}}$, co pozwala uzyskać wartość z przedziału $(0, 1)$ interpretowaną jako prawdopodobieństwo przynależności tekstu do danej klasy (np. prawdopodobieństwo, że recenzja jest pozytywna).

4. Klasyfikacja binarna i wieloklasowa, SOFTMAX

- **Klasyfikacja binarna:** Podział na dokładnie dwie wykluczające się klasy (0 lub 1).
- **Klasyfikacja wieloklasowa:** Przypisywanie tekstu do jednej z wielu możliwych klas (np. sport, biznes, polityka, rozrywka).
- **Funkcja SOFTMAX:** Jest to uogólnienie funkcji sigmoidalnej dla wielu klas. Przyjmuje na wejściu wektor surowych wyników (logitów) z sieci i przekształca je w rozkład prawdopodobieństwa (wszystkie wartości są z przedziału $(0,1)$ i sumują się do 1).
- Matematycznie w modelach językowych mapuje ona reprezentację kontekstową na przestrzeń całego słownika: $\text{vec}(p) = \text{operatorname{softmax}}(f(\text{vec}(c)))$.

5. Wielowarstwowe sieci feedforward na tekście: klasyfikacja i regresja

- Klasyczne sieci neuronowe (gęste / MLP), w których informacja przepływa tylko w jednym kierunku – od warstwy wejściowej, przez warstwy ukryte, do warstwy wyjściowej.

- **W klasyfikacji:** Ostatnia warstwa posiada liczbę neuronów równą liczbie klas i kończy się funkcją *Softmax* (lub *Sigmoid* dla klasyfikacji binarnej).
- **W regresji:** Ostatnia warstwa posiada zazwyczaj jeden neuron z liniową funkcją aktywacji (brak obcięcia wartości), zwracający ciągłą wartość liczbową.

6. Podstawy normalizacji tekstu (tokenizacja, lematyzacja)

- **Tokenizacja:** Proces dzielenia ciągłego tekstu na mniejsze jednostki, zwane tokenami (mogą to być całe wyrazy, znaki interpunkcyjne lub części słów, tzw. jednostki BPE/subwords).
- **Lematyzacja:** Sprowadzanie wyrazów odmienionych w tekście do ich formy podstawowej, słownikowej (tzw. lematu, np. "będziemy" $\rightarrow$ "być", "kotami" $\rightarrow$ "kot").

7. Pojęcie one-hot encoding dla wyrazów, pojęcie słownika (vocabulary)

- **Słownik (Vocabulary):** Zbiór wszystkich unikalnych tokenów/słów występujących w korpusie treningowym, gdzie każdemu elementowi przypisany jest unikalny indeks liczbowy.
- **One-hot encoding:** Sposób reprezentacji słowa w postaci rzadkiego wektora o długości równej rozmiarowi słownika ($|V|$). Wektor ten zawiera samą wartość 0 na wszystkich pozycjach, z wyjątkiem jednej pozycji odpowiadającej indeksowi danego słowa, gdzie znajduje się wartość 1.

8. Rzadkie reprezentacje wektorowe: tf, tf-idf

- **TF (Term Frequency):** Częstość występowania danego słowa w obrębie jednego dokumentu.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** Statystyka wagowa informująca o tym, jak ważne jest słowo dla danego dokumentu w kontekście całego korpusu. Równoważy częste występowanie słowa w dokumencie (TF) z jego powszechnością w innych dokumentach (IDF). Słowa występujące niemal w każdym dokumencie (np. spójniki "i", "w", "że") otrzymują niską wagę IDF.

9. „Hashing trick”

- Technika mapowania słów/cech o zmiennej długości na wektory o stałym, z góry określonym rozmiarze za pomocą funkcji skrótu (hashującej).
- **Zaleta:** Eliminuje potrzebę utrzymywania i budowania gigantycznego słownika w pamięci – indeks pozycji w wektorze jest wyliczany bezpośrednio ze słowa.
- **Wada:** Może prowadzić do kolizji (dwa różne słowa zostaną przypisane do tej samej pozycji).

10. Word2vec: użycie i uczenie, własności wektorów

- Technika tworzenia gęstych reprezentacji wektorowych słów (embeddingów).

- **Uczenie:** Opiera się na hipotezie dystrybucyjnej (słowa występujące w podobnych kontekstach mają podobne znaczenie). Uczy się je poprzez przewidywanie słowa na podstawie otaczających słów (CBOW) lub otaczających słów na podstawie słowa centralnego (Skip-gram).
- **Własności wektorów:** Wektory te wychwytyują relacje semantyczne i syntaktyczne. Pozwalają na wykonywanie operacji arytmetycznych na znaczeniach, np. popularne: $\vec{w}_{\text{król}} - \vec{w}_{\text{mężczyzna}} + \vec{w}_{\text{kobieta}} \approx \vec{w}_{\text{królowa}}$.

11. Sposoby agregowania wektorów słów do wektorów tekstów

Aby reprezentować całe zdanie lub dokument za pomocą embeddingów, pojedyncze wektory słów można agregować m.in. poprzez:

- Średnią arytmetyczną wektorów wszystkich słów w dokumencie (Continuous Bag of Words dla tekstu).
- Sumowanie wektorów słów.
- Średnią ważoną (np. wagami TF-IDF, dając większe znaczenie słowom kluczowym).

12. Mierzenie podobieństwa między słowami/tekstami, podobieństwo kosinusowe

- **Podobieństwo kosinusowe (Cosine similarity):** Miara oceniająca bliskość dwóch wektorów w przestrzeni wielowymiarowej poprzez wyznaczenie kosinusa kąta między nimi.
- Wzór: $\text{sim}(A, B) = \frac{A \cdot B}{|A| |B|}$. Przyjmuje wartość 1\$, gdy wektory mają ten sam kierunek (są maksymalnie podobne semantycznie), oraz 0\$, gdy są ortogonalne (brak podobieństwa).

13. Modele Transformer: typy architektur i realizowanych zadań

Modele oparte na architekturze Transformer dzielimy ze względu na strukturę na trzy główne typy:

1. **Encoder-only (Tylko koder):** Przetwarzają tekst dwukierunkowo (z uwzględnieniem lewego i prawego kontekstu jednocześnie). Idealne do zadań klasyfikacji, ekstrakcji informacji i etykietowania sekwencji. Przykłady: **BERT, RoBERTa**.
2. **Decoder-only (Tylko dekoder):** Przetwarzają tekst asymetrycznie, zazwyczaj ściśle od lewej do prawej (autoregresyjnie). Służą do generowania tekstu, systemów dialogowych czy streszczania. Przykłady: **GPT, GPT-2, GPT-3**.
3. **Encoder-decoder (Koder-dekoder):** Podejście sekwencja-do-sekwencji (generatywne). Koder przetwarza tekst wejściowy, a dekoder generuje odpowiedź. Stosowane w tłumaczeniach maszynowych oraz ogólnych zadaniach zamiany tekstu na tekst. Przykłady: **T5, mT5**.

14. Szczegóły architektury encoder-only, przykładowe modele

- Modele te składają się z bloku koder z oryginalnej architektury Transformer. Kluczowym

elementem jest mechanizm **Self-Attention** (własnej uwagi), który pozwala każdemu tokenowi w sekwencji "patrzeć" na wszystkie inne tokeny w tym samym zdaniu (zarówno po lewej, jak i po prawej stronie).

- **Mechanizm Scaled Dot-Product Attention:**

Atencję interpretuje się jako miękkie odpytywanie bazy danych składającej się z par klucz (\$K\$) - wartość (\$V\$) za pomocą zapytania (\$Q\$).

Wzór na atencję z dodatkową normalizacją przez rozmiar klucza \$d_k\$ to:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Pochodzenie Q, K, V:** W pierwszej warstwie macierze te powstają z przemnożenia embeddingów tokenów \$E\$ przez macierze wag (\$W^Q, W^K, W^V\$):

- \$Q = EW^Q\$, \$K = EW^K\$, \$V = EW^V\$.

W kolejnych warstwach zamiast wejściowych embeddingów \$E\$ używa się wyjścia z warstwy poprzedniej.

- **Zastosowanie w ekstrakcji informacji:** Do reprezentacji wyjściowych z warstw encoder-only można w prosty sposób dołączyć dedykowaną głowicę klasyfikacyjną realizującą etykietowanie sekwencji (np. rozpoznawanie encji nazwanych - NER).

- **Przykłady modeli:** BERT, RoBERTa (w tym Polish RoBERTa).

15. Architektura modelu BERT, sposób uczenia

- **Architektura:** BERT (Bidirectional Encoder Representations from Transformers) to czysty model typu *encoder-only*.

- **Sposób uczenia (Pretrenowanie - Pretraining):** BERT uczony jest na ogromnych, nieskażonych ludzkich korpusach tekstowych w sposób nienadzorowany (samo-nadzorowany) za pomocą zadania **Maskowania słów (Masked Language Modeling - MLM)**.

- **Schemat maskowania w BERT:**

Losowo wybiera się **15% wyrazów** z tekstu do odgadnięcia przez model. Następnie wybrane słowa podlegają modyfikacji według proporcji:

- **80%** z nich jest zastępowanych specjalnym tokenem maski ([MASK]).
- **10%** jest zastępowanych zupełnie innym, przypadkowym wyrazem ze słownika.
- **10%** pozostawia się bez zmian (ale model i tak musi je odgadnąć).

- Dzięki temu model zmuszony jest do głębokiego i dwukierunkowego rozumienia relacji między słowami (kontekstu lewego i prawego jednocześnie).